

# State Estimation

# What is state estimation?

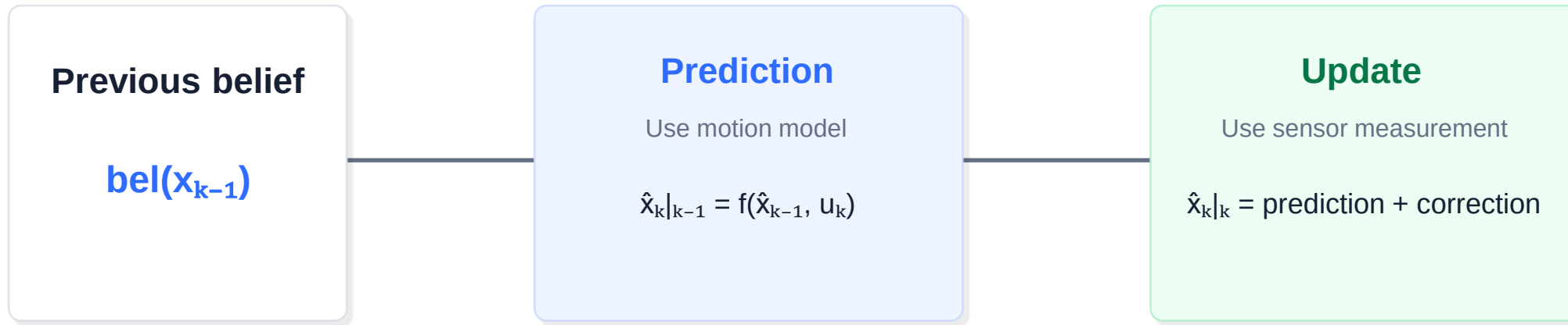
State estimation is the process of estimating the hidden true state of a system by combining a motion model, sensor measurements, and uncertainty.



Example robot state:  $x = [\text{position\_x}, \text{position\_y}, \text{heading}, \text{velocity}]$

A good estimator answers: Where am I? How confident am I?

# Most estimators follow prediction and update



Prediction usually increases uncertainty because motion is imperfect. Update usually decreases uncertainty because new sensor data gives evidence.

# Compact belief: one mean and one covariance

**Kalman Filter represents belief as a Gaussian distribution.**

covariance = uncertainty width

- **Assumption** linear model + Gaussian noise
- **Output** best estimate  $\hat{x}$  and covariance  $P$
- **Strength** fast and mathematically elegant
- **Limitation** less suitable for strongly nonlinear or multi-modal uncertainty

Trust prediction or trust sensor? Kalman gain decides.

# Solved example: one-dimensional position estimate

## Prediction

$$\hat{x}^- = \hat{x} + u \quad P^- = P + Q$$

## Update

$$K = P^- / (P^- + R)$$

$$\hat{x} = \hat{x}^- + K(z - \hat{x}^-)$$

$$P = (1 - K)P^-$$

Interpretation: K is high when the measurement is reliable, so the correction is stronger.

## Numerical example

**Given**  $\hat{x} = 0.00$ ,  $P = 1.00$ ,  $u = 1.00$ ,  $Q = 0.20$

**Measure**  $z = 1.30$ ,  $R = 0.30$

**Predict**  $\hat{x}^- = 1.00$ ,  $P^- = 1.20$

**Gain**  $K = 1.20 / (1.20 + 0.30) = 0.80$

**Update**  $\hat{x} = 1.00 + 0.80(1.30 - 1.00) = 1.24$

**Uncertainty**  $P = (1 - 0.80)(1.20) = 0.24$

Result: estimate moves toward the measurement, uncertainty drops from 1.20 to 0.24.

# Kalman filtering extends to nonlinear robot localization

In the simulation, the state contains the robot pose, velocity, and landmark positions.

## State vector

$x = [\text{robot } x, \text{robot } y, \text{heading } \psi, \text{speed } V, \text{landmark}_1, \dots, \text{landmark}_5]$

The robot and map are estimated together.

## Nonlinear motion

$$x_{k+1} = x_k + V \cos(\psi)\Delta t$$

$$y_{k+1} = y_k + V \sin(\psi)\Delta t$$

EKF linearizes this model using Jacobians.

## Sensor model

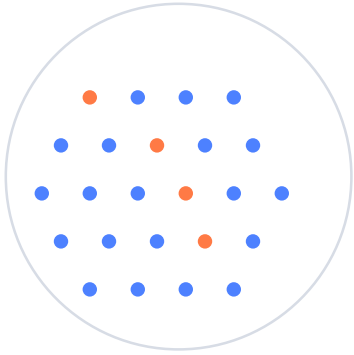
Measurements include heading, speed, landmark bearing, and landmark range.

R controls sensor noise. Higher R means the filter trusts measurements less.

This is why changing sensor accuracy changes the size of the red uncertainty ellipses in the results.

# Sample-based belief: many guesses with weights

Particle Filter does not force belief into one Gaussian. It represents many possible robot states.



particles = possible states

- 1 Initialize particles
- 2 Predict motion
- 3 Weight by sensor likelihood
- 4 Resample high-weight particles

Key intuition: bad guesses disappear, good guesses survive.

# Solved example: measurement weighting and resampling

Assume a 1D position measurement  $z = 1.0$  with sensor standard deviation  $\sigma = 0.2$ .

## Particle weights

| particle | x   | raw likelihood | normalized w |
|----------|-----|----------------|--------------|
| 1        | 0.2 | 0.0003         | 0.0002       |
| 2        | 0.7 | 0.3247         | 0.1968       |
| 3        | 1.0 | 1.0000         | 0.6062       |
| 4        | 1.3 | 0.3247         | 0.1968       |
| 5        | 1.9 | 0.00004        | 0.00002      |

$$w_i \propto \exp(-1/2 \cdot (x_i - z)^2 / \sigma^2)$$

## Resampling intuition



before: spread guesses

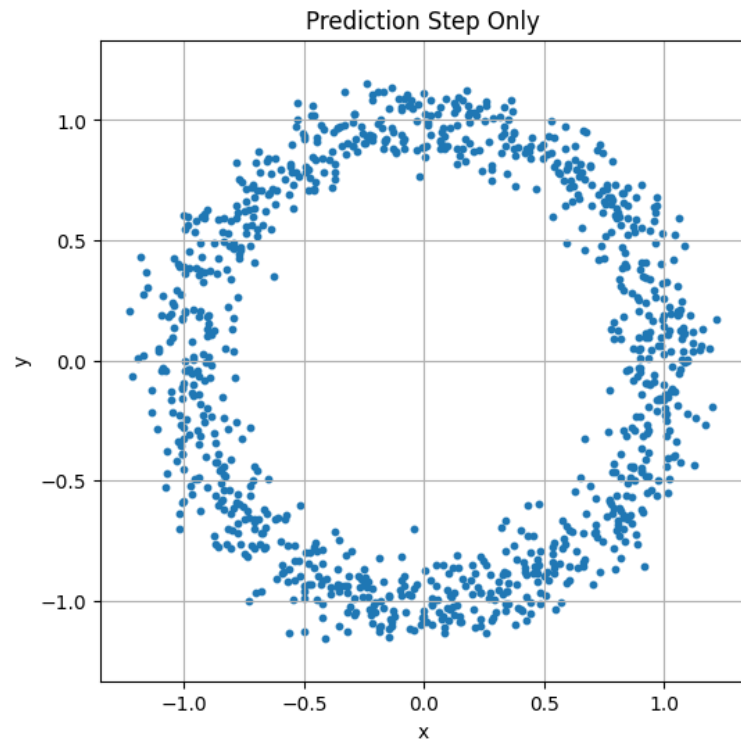


after: concentrated near  $z = 1.0$

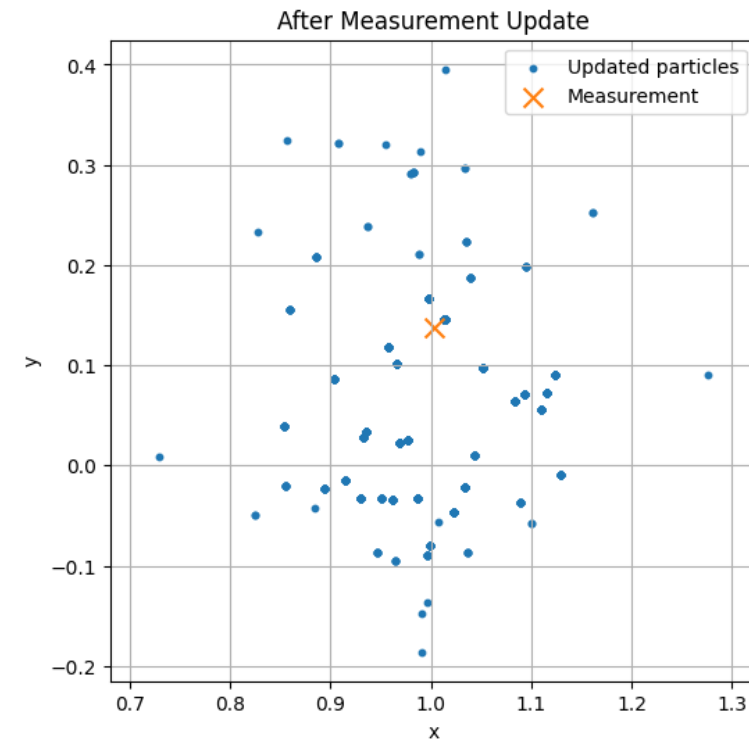
Particle 3 dominates because it best explains the measurement.



# Nonlinear system: prediction only versus measurement update



Prediction only

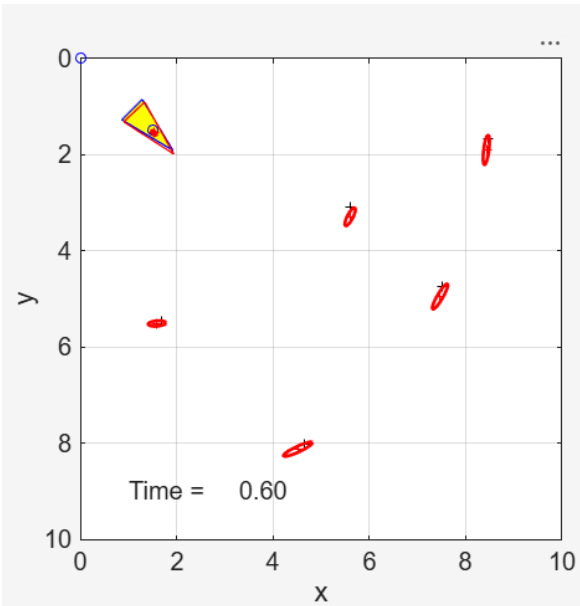


After measurement update

Analysis: heading uncertainty creates a circular prediction cloud. The measurement update assigns high weights near the observed state and resampling collapses the cloud around the measurement.

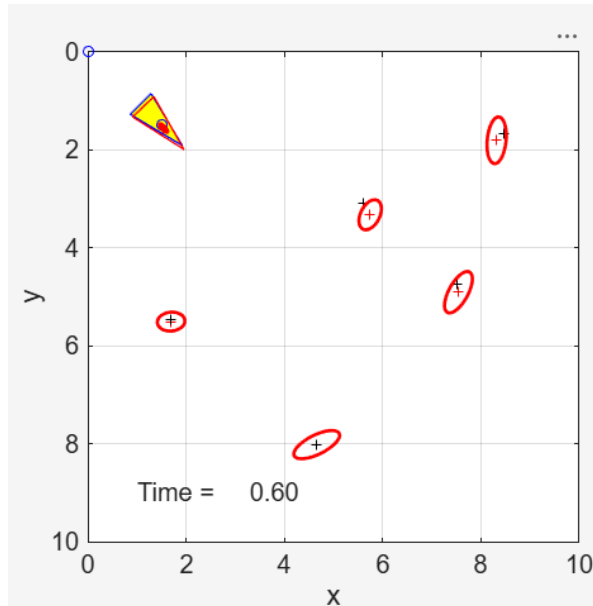
# Sensor accuracy changes the confidence ellipses

Better sensor



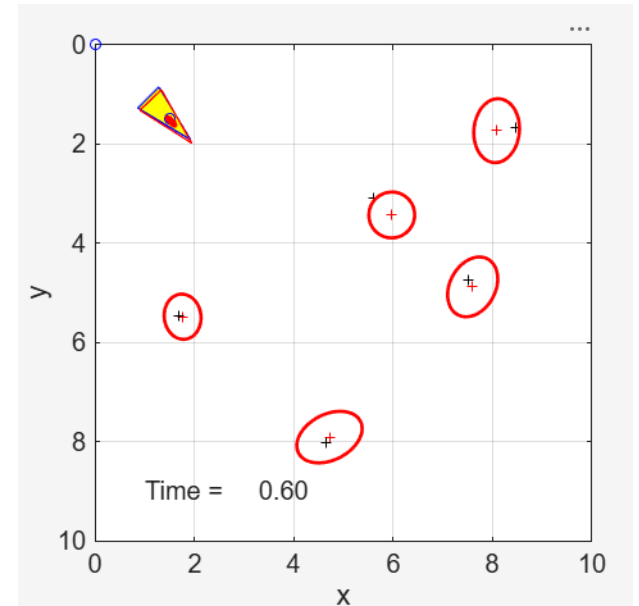
Small ellipses: high confidence

Original sensor



Medium ellipses: baseline confidence

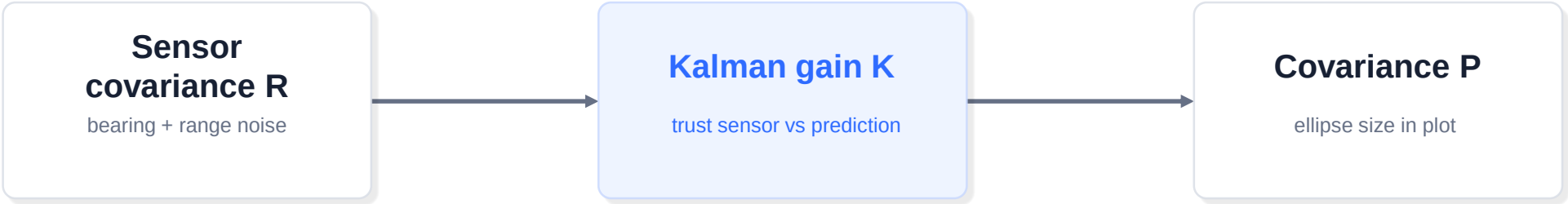
Worse sensor



Large ellipses: low confidence

Red ellipses show 95% confidence regions for the estimated robot and landmarks. Larger ellipses mean higher covariance and lower certainty.

# Why the three EKF-SLAM outputs differ



|                          |                            |                                     |                                              |
|--------------------------|----------------------------|-------------------------------------|----------------------------------------------|
| ● <b>Better sensor</b>   | Lower measurement noise    | Filter trusts measurements more     | Tighter ellipses, clearer landmark estimates |
| ● <b>Original sensor</b> | Moderate measurement noise | Balanced prediction and measurement | Medium ellipses, expected baseline result    |
| ● <b>Worse sensor</b>    | Higher measurement noise   | Filter trusts measurements less     | Larger ellipses, uncertainty remains high    |

**Main takeaway: the filter is not just estimating position. It is also estimating how much it should trust that position.**

# Kalman Filter and Particle Filter solve the same problem differently

## Kalman Filter / EKF

|                  |                                   |
|------------------|-----------------------------------|
| <b>Belief</b>    | mean + covariance                 |
| <b>Best when</b> | uncertainty is close to Gaussian  |
| <b>Speed</b>     | fast and efficient                |
| <b>Weakness</b>  | struggles with multi-modal belief |

## Particle Filter

|                  |                                       |
|------------------|---------------------------------------|
| <b>Belief</b>    | many weighted particles               |
| <b>Best when</b> | nonlinear or non-Gaussian uncertainty |
| <b>Speed</b>     | more computation                      |
| <b>Weakness</b>  | needs enough particles                |

Both algorithms follow prediction → measurement update. They mainly differ in how they represent uncertainty.

# KITTI Dead Reckoning and EKF State Estimation

Goal: move from small simulated examples to a real driving dataset using IMU-only dead reckoning and IMU + GPS EKF fusion.

## Why this belongs here

It directly applies the same state-estimation idea: predict from motion, correct with measurements, and analyze uncertainty and drift.

## What changes

The problem becomes real-data based: timestamps, coordinate conversion, sensor quality, bias, and evaluation against a reference trajectory.

# KITTI Raw is the correct release for IMU/GPS experiments

Use the raw synced drive, not only the odometry benchmark, because the raw release contains OXTS GPS/IMU packets.

| Item          | Choice in this report      | Reason                                                        |
|---------------|----------------------------|---------------------------------------------------------------|
| Release       | KITTI Raw synced           | Contains synchronized GPS/IMU OXTS files                      |
| Example drive | 2011_09_26_drive_0019_sync | Common pykitti example drive and easy to reproduce            |
| Reference     | OXTS-derived local pose    | Usable baseline reference, but not independent mocap truth    |
| Frame         | Local ENU-like x-y frame   | Converts latitude/longitude into meters for plotting and RMSE |

## Important reproducibility note

KITTI Raw downloads require registration and the raw sequence zip was not attached here. Therefore, the report gives runnable code and expected analysis, but does not invent numeric RMSE values.

# From KITTI OXTS packets to estimator-ready data



Minimum checks: dt is nonzero after the first sample, yaw is unwrapped, GPS position is in meters, and missing or low-quality OXTS entries are handled.

A common mistake is treating every GPS update as equally reliable. Adaptive R based on reported position accuracy makes the EKF more robust.

# Two baselines: IMU-only dead reckoning and IMU + GPS EKF

## Baseline A: IMU-only dead reckoning

State:  $[x, y, \theta, v]$

Inputs: forward acceleration  $a_f$  and yaw rate  $w_z$

Prediction: integrate acceleration and heading over time

Expected issue: small IMU bias becomes large position drift after integration.

## Baseline B: IMU + GPS EKF

State:  $[x, y, \theta, v]$  or  $[x, y, \theta, v, b_g, b_a]$

Prediction: IMU-driven motion model

Update: GPS x-y position measurement

Expected benefit: GPS correction keeps position error bounded.

## Core EKF idea

**Prediction from IMU motion model + correction from GPS position measurement = improved state estimate**

Bias-augmented EKF is the stronger version because gyro and accelerometer bias are the main drivers of long-term drift.



# How to turn the experiment into defensible results

| Output                     | What it shows                                      |
|----------------------------|----------------------------------------------------|
| Trajectory plot            | Visual drift: reference vs IMU-only vs EKF         |
| Error vs time plot         | Whether error grows unbounded or stays corrected   |
| RMSE [m]                   | Average position disagreement with OXTS reference  |
| Final error [m]            | Endpoint drift after the full sequence             |
| Max error [m]              | Worst-case deviation during the drive              |
| KITTI-style relative error | Subsequence performance over 100 to 800 m segments |

## How to phrase the reference carefully

Report the numbers as agreement with the OXTS-derived reference, not as perfect ground truth. This avoids overstating precision.

# What should differ between dead reckoning and EKF

| Method                  | Expected visual result                                           | Expected numeric result                          | Reason                                                        |
|-------------------------|------------------------------------------------------------------|--------------------------------------------------|---------------------------------------------------------------|
| IMU-only dead reckoning | Trajectory drifts away, especially after turns and speed changes | RMSE and final error grow quickly                | Noise and bias are integrated into velocity and then position |
| IMU + GPS EKF           | Trajectory follows the OXTS path much more closely               | Lower RMSE, lower final error, bounded max error | GPS updates correct accumulated inertial drift                |
| Bias-augmented EKF      | Similar path shape but usually more stable heading and velocity  | Lower or more stable error than no-bias EKF      | Estimated gyro and accel bias reduce systematic drift         |

**Main analytical point: dead reckoning explains why state estimation is necessary, and EKF shows how sensor fusion reduces uncertainty and drift.**

# Where this baseline can go next

| Improvement                              | Why it helps                                                          |
|------------------------------------------|-----------------------------------------------------------------------|
| Add explicit IMU bias states             | Reduces systematic heading and velocity drift                         |
| Move from EKF to error-state EKF or IEKF | Better handles nonlinear pose geometry and improves consistency       |
| Adaptive or learned covariance tuning    | Lets Q and R respond to changing motion and GPS quality               |
| Add camera, wheel odometry, or LiDAR     | Provides geometric constraints that inertial/GPS alone cannot provide |
| Measurement gating and outlier rejection | Prevents low-quality GPS packets from destabilizing the update        |

Literature pattern: stronger methods improve by modeling bias, using more sensors, and treating sensor uncertainty more carefully.

This connects naturally to your larger project direction: EKF/UKF baselines first, then IEKF/InEKF and adaptive noise tuning.

# How to present this as the ending section

## 1. Weak baseline

IMU-only dead reckoning intentionally drifts. It shows why sensor fusion is necessary.

## 2. Classical baseline

IMU + GPS EKF corrects drift and gives a defensible baseline estimator.

## 3. Research extension

Bias states, IEKF/InEKF, adaptive noise tuning, and multimodal fusion make the estimator more robust.

The KITTI experiment turns state estimation from a formula into a real robotics problem..

Specifically, noisy motion creates drift and the filter design determines how trustworthy the final trajectory is.